

claude-windows / 7fce5410-4c57-4ec9-a054-be898bda22da

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7fce5410-4c57-4ec9-a054-be898bda22da\subagents\workflows\wf_c81dbe7a-7ff\agent-a18bcf92b99884915.jsonl`  
- SHA-256: `33fc94b8d918bca115d010148a0e331a316e118a60f1dc85f54b470ca8cbdf68`  
- Source modified: `2026-06-25T16:57:04+00:00`  
- Imported at: `2026-07-05T16:48:26+00:00`  
- project: `wf_c81dbe7a-7ff`  
- session_id: `7fce5410-4c57-4ec9-a054-be898bda22da`
```

Transcript

1. user (2026-06-25T16:56:32.125Z)

Adversarially verify this PR-review finding for PR #380 (branch o1-p11/gemini-remote-file-gc is checked out; read the real code with `git diff origin/master...HEAD` + Read the files). Be skeptical ? default to `isReal=false` / `adjustedSeverity=INVALID` if the finding is wrong, already handled by the code, or a non-issue. Do NOT re-run CI.

Finding [NIT] Misleading test comment references `_safe_delete`, which this code path does not use
Location: `tests/test_provider_gemini.py`
(`test_cleanup_orphaned_files_per_file_delete_failure_is_swallowed`, inline comment "`_safe_delete swallows it`")

Detail: The test comment says "First delete raises, second succeeds ? `_safe_delete` swallows it."
But `cleanup_orphaned_files()` calls `self.client.files.delete()` directly inside its own per-file try/except (`gemini.py:78-81`); it never calls `_safe_delete`. The behavior under test is correct and the assertion is right (`deleted==1`), only the comment is inaccurate ? it could mislead a future reader into thinking `_safe_delete` is on this path.

Suggested fix: Reword the comment to "First delete raises, second succeeds ? the per-file except in `cleanup_orphaned_files` swallows it (only the success is counted)."

Return `isReal` + `adjustedSeverity` (`INVALID` if not real) + one-paragraph reasoning grounded in the actual code.

2. assistant (2026-06-25T16:56:34.909Z)

I'll verify this finding by examining the actual code in PR #380.

3. user (2026-06-25T16:56:36.459Z)

f00b71e feat(01/P11): Gemini remote-file GC ? startup orphan sweep
o1-p11/gemini-remote-file-gc

4. user (2026-06-25T16:56:36.979Z)

```
backend/main.py          | 25 ++++++++  
backend/providers/gemini.py | 43 ++++++++  
tests/test_provider_gemini.py | 104 ++++++++  
3 files changed, 172 insertions(+)
```

5. assistant (2026-06-25T16:56:39.877Z)

Let me read the actual `gemini.py` code path and the test.

6. user (2026-06-25T16:56:41.040Z)

```
diff --git a/backend/providers/gemini.py b/backend/providers/gemini.py
```

```

index 2ea0834..428bca8 100644
--- a/backend/providers/gemini.py
+++ b/backend/providers/gemini.py
@@ -2,6 +2,7 @@ import json
import logging
import os
import time
+from datetime import datetime, timedelta, timezone
from typing import Tuple

from google import genai
@@ -40,6 +41,48 @@ class GeminiProvider(AIVideoProvider):
    except Exception:
        pass

+
+ def cleanup_orphaned_files(self, grace_sec: int = 3600) -> int:
+     """Sweep and delete video files orphaned on the Gemini File API by a previous
+     crashed run (a hard process death between ``files.upload`` and ``_safe_delete``
+     leaves the uploaded clip behind forever ? the per-call cleanup only covers paths
+     that reach a ``finally``/except).
+
+     Best-effort: lists every file, deletes any whose ``mime_type`` is a video AND whose
+     creation is older than ``grace_sec`` (so we never race a concurrent worker mid-upload).
+     Never raises; returns the count deleted. Intended to run once at startup.
+
+     Args:
+         grace_sec: only delete files older than this (default 1h). Protects a file a
+         live worker may still be processing.
+     """
+     cutoff = datetime.now(timezone.utc) - timedelta(seconds=grace_sec)
+     deleted = 0
+     try:
+         files = list(self.client.files.list())
+     except Exception as e:
+         logger.warning("[Gemini GC] failed to list remote files (skipping sweep): %s", e)
+         return 0
+     for f in files:
+         try:
+             mime = getattr(f, "mime_type", "") or ""
+             if not mime.startswith("video/"):
+                 continue
+             # f.create_time is an RFC3339 string on the File API.
+             ct = getattr(f, "create_time", None)
+             if ct is None:
+                 # No timestamp ? can't prove it's stale; leave it (safe default).
+                 continue
+             created = datetime.fromisoformat(str(ct).replace("Z", "+00:00"))
+             if created > cutoff:
+                 continue # too recent ? a live worker may own it
+             self.client.files.delete(name=f.name)
+             deleted += 1
+         except Exception as e:
+             logger.debug("[Gemini GC] skipping a file during sweep: %s", e)
+     if deleted:
+         logger.info("[Gemini GC] deleted %d orphaned video file(s) older than %ds.",
deleted, grace_sec)
+     return deleted
+
+
+ def analyze_video(self, video_path: str, prompt: str,
+                 chunk_duration: float = 0.0,
+                 label: str = "") -> Tuple[list, dict]:

```

7. user (2026-06-25T16:56:41.313Z)

diff --git a/tests/test_provider_gemini.py b/tests/test_provider_gemini.py

```

Index 141af81..5df2ef4 100644
--- a/tests/test_provider_gemini.py
+++ b/tests/test_provider_gemini.py
@@ -101,3 +101,107 @@ def test_gemini_provider_upload_fail():
     provider = GeminiProvider(api_key="dummy_api_key", model_name="gemini-2.5-flash")
     segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt")
     assert len(segments) == 0
+
+
+
+# --- 01/P11: Gemini remote-file GC (startup orphan sweep) ---
+
+def _file_mock(name, mime, create_time_iso, state="ACTIVE"):
+    f = MagicMock()
+    f.name = name
+    f.mime_type = mime
+    f.create_time = create_time_iso
+    f.state = MagicMock()
+    f.state.name = state
+    return f
+
+
+def test_cleanup_orphaned_files_deletes_stale_videos_only():
+    """Deletes video files older than the grace window; leaves non-videos and fresh files."""
+    from datetime import datetime, timedelta, timezone
+    stale = (datetime.now(timezone.utc) - timedelta(hours=2)).isoformat()
+    fresh = (datetime.now(timezone.utc) - timedelta(minutes=5)).isoformat()
+    files = [
+        _file_mock("old-vid", "video/mp4", stale),          # stale video -> deleted
+        _file_mock("fresh-vid", "video/quicktime", fresh), # fresh video -> kept (grace)
+        _file_mock("old-img", "image/png", stale),          # non-video -> kept
+        _file_mock("notts", "video/mp4", None),             # no timestamp -> kept (safe)
+    ]
+    with patch("backend.providers.gemini.genai"):
+        provider = GeminiProvider(api_key="k", model_name="m")
+        provider._client = MagicMock()
+        provider._client.files.list.return_value = iter(files)
+        deleted = provider.cleanup_orphaned_files(grace_sec=3600)
+    assert deleted == 1
+    provider._client.files.delete.assert_called_once_with(name="old-vid")
+
+
+def test_cleanup_orphaned_files_list_failure_is_swallowed():
+    """A list() failure must not raise ? the sweep is best-effort and must not block startup."""
+    with patch("backend.providers.gemini.genai"):
+        provider = GeminiProvider(api_key="k", model_name="m")
+        provider._client = MagicMock()
+        provider._client.files.list.side_effect = RuntimeError("network")
+        assert provider.cleanup_orphaned_files() == 0
+
+
+def test_cleanup_orphaned_files_per_file_delete_failure_is_swallowed():
+    """A delete() failure on one file must not abort the sweep."""
+    from datetime import datetime, timedelta, timezone
+    stale = (datetime.now(timezone.utc) - timedelta(hours=2)).isoformat()
+    files = [
+        _file_mock("a", "video/mp4", stale),
+        _file_mock("b", "video/mp4", stale),
+    ]
+    with patch("backend.providers.gemini.genai"):
+        provider = GeminiProvider(api_key="k", model_name="m")
+        provider._client = MagicMock()
+        provider._client.files.list.return_value = iter(files)
+        # First delete raises. second succeeds ? safe delete swallows it.

```



```

39         try:
40             self.client.files.delete(name=video_file.name)
41         except Exception:
42             pass
43
44     def cleanup_orphaned_files(self, grace_sec: int = 3600) -> int:
45         """Sweep and delete video files orphaned on the Gemini File API by a previous
46         crashed run (a hard process death between ``files.upload`` and ``_safe_delete``
47         leaves the uploaded clip behind forever ? the per-call cleanup only covers paths
48         that reach a ``finally``/except).
49
50         Best-effort: lists every file, deletes any whose ``mime_type`` is a video AND
51         whose
52         creation is older than ``grace_sec`` (so we never race a concurrent worker mid-
53         upload).
54         Never raises; returns the count deleted. Intended to run once at startup.
55
56         Args:
57             grace_sec: only delete files older than this (default 1h). Protects a file a
58             live worker may still be processing.
59         """
60         cutoff = datetime.now(timezone.utc) - timedelta(seconds=grace_sec)
61         deleted = 0
62         try:
63             files = list(self.client.files.list())
64         except Exception as e:
65             logger.warning("[Gemini GC] failed to list remote files (skipping sweep):
66             %s", e)
67             return 0
68         for f in files:
69             try:
70                 mime = getattr(f, "mime_type", "") or ""
71                 if not mime.startswith("video/"):
72                     continue
73                 # f.create_time is an RFC3339 string on the File API.
74                 ct = getattr(f, "create_time", None)
75                 if ct is None:
76                     # No timestamp ? can't prove it's stale; leave it (safe default).
77                     continue
78                 created = datetime.fromisoformat(str(ct).replace("Z", "+00:00"))
79                 if created > cutoff:
80                     continue # too recent ? a live worker may own it
81                 self.client.files.delete(name=f.name)
82                 deleted += 1
83             except Exception as e:
84                 logger.debug("[Gemini GC] skipping a file during sweep: %s", e)
85         if deleted:
86             logger.info("[Gemini GC] deleted %d orphaned video file(s) older than %ds.",
87             deleted, grace_sec)
88         return deleted
89
90     def analyze_video(self, video_path: str, prompt: str,
91                       chunk_duration: float = 0.0,
92                       label: str = "") -> Tuple[list, dict]:
93         log_prefix = f"Gemini {label}" if label else "Gemini"

```

10. assistant (2026-06-25T16:56:47.561Z)

The finding is confirmed by the code. Let me verify the exact test comment text.

11. user (2026-06-25T16:56:48.835Z)

```

143         provider._client = MagicMock()
144         provider._client.files.list.side_effect = RuntimeError("network")
145         assert provider.cleanup_orphaned_files() == 0

```

```

146
147
148 def test_cleanup_orphaned_files_per_file_delete_failure_is_swallowed():
149     """A delete() failure on one file must not abort the sweep."""
150     from datetime import datetime, timedelta, timezone
151     stale = (datetime.now(timezone.utc) - timedelta(hours=2)).isoformat()
152     files = [
153         _file_mock("a", "video/mp4", stale),
154         _file_mock("b", "video/mp4", stale),
155     ]
156     with patch("backend.providers.gemini.genai"):
157         provider = GeminiProvider(api_key="k", model_name="m")
158         provider._client = MagicMock()
159         provider._client.files.list.return_value = iter(files)
160         # First delete raises, second succeeds ? _safe_delete swallows it.
161         provider._client.files.delete.side_effect = [RuntimeError("boom"), None]
162         deleted = provider.cleanup_orphaned_files(grace_sec=3600)

```

12. assistant (2026-06-25T16:56:56.440Z)

The finding is verified against the actual code. The facts check out:

1. The test at line 160 contains the inline comment ``# First delete raises, second succeeds ? _safe_delete swallows it.``
2. The code path under test, ``cleanup_orphaned_files()`` (gemini.py:44-84), calls ``self.client.files.delete(name=f.name)`` directly at line 78, inside its own per-file ``try/except`` block (lines 66-81) ? it does NOT call ``_safe_delete()``.
3. ``_safe_delete()`` (lines 37-42) is a separate method used by ``analyze_video``'s per-call cleanup, not by the GC sweep. It's even referenced in the docstring of ``cleanup_orphaned_files`` as the thing the GC compensates for after a crash.
4. The test's assertion logic is correct: ``deleted == 1`` because the first ``delete()`` raises (caught by the per-file except, file not counted) and the second succeeds (counted). The behavior under test and the assertion are right ? only the comment's attribution to ``_safe_delete`` is inaccurate.

The comment could indeed mislead a reader into thinking ``_safe_delete`` is on this path, when the actual error-swallowing is done by the inline ``except Exception as e`` at gemini.py:80. This is a genuine but purely cosmetic documentation inaccuracy in test code ? no functional impact, no test correctness issue. NIT is the appropriate severity.

13. user (2026-06-25T16:57:01.139Z)

Structured output provided successfully

14. assistant (2026-06-25T16:57:04.562Z)

Verified the finding against the real code. It is a real but cosmetic issue (NIT). The comment at ``tests/test_provider_gemini.py:160`` misattributes the error-swallowing to ``_safe_delete``, but ``cleanup_orphaned_files()`` (``backend/providers/gemini.py:44-84``) calls ``self.client.files.delete()`` directly at line 78 inside its own per-file ``except`` (line 80) and never touches ``_safe_delete``. The test's assertion (``deleted == 1``) is correct; only the comment is inaccurate.